

REAK — COMPLETE PRODUCTION DEVELOPMENT COMMAND

You are the lead engineering orchestrator for the REAK platform.

The planning/specification stage is complete.

Your job now is to **actually build the complete REAK application in the current repository**.

Do not merely explain what should be built.

Do not generate a theoretical architecture.

Do not give me a tutorial.

Inspect the repository, understand the existing code, then implement the application end-to-end.

1. SOURCE OF TRUTH

Use the following priority order:

Priority 1 — REAK Product Requirements

The approved REAK architecture and product specification supplied in this conversation is the primary source of truth.

It defines:

- Public Website
- Member Portal
- Admin Portal
- Membership
- Property Exchange
- Property Listings
- Client Requirements/Demands
- Bidirectional Matching
- Collaboration
- Contact Privacy
- CMS
- RBAC
- RLS
- Feature Flags

- Audit Logging
- Storage
- Nepal Location Hierarchy
- Area Units
- Notifications
- Security
- Accessibility
- Performance
- SEO

Priority 2 — Existing Repository

The existing codebase is authoritative for:

- existing working functionality
- existing project configuration
- existing environment conventions
- existing integrations
- existing reusable components

Do not destroy useful existing work.

Priority 3 — Enterprise `.claude` Framework

Use the project's `.claude` agents, workflows, skills and contexts.

Your framework is designed for deterministic full-stack execution.

Use it.

2. FIRST ACTION — DO NOT CODE BLINDLY

Before modifying anything, inspect the repository.

Determine:

- current working directory
- framework
- package manager
- frontend framework
- backend architecture
- database
- Supabase configuration
- existing routes
- existing components

- existing authentication
- existing migrations
- existing tests
- environment files
- `.claude/` framework
- MCP configuration
- installed skills
- existing documentation

Run appropriate repository inspection commands.

Do NOT assume:

- Next.js
- React Router
- Supabase
- folder names
- package manager
- deployment provider

until verified from the repository.

If the existing project already uses the approved REAK technology stack, preserve it.

If the project differs, determine whether migration is actually necessary before changing architecture.

3. USE THE ENTERPRISE WORKFLOW

Use the existing:

`full-stack-feature`

workflow for the implementation.

Use the appropriate agents through the `.claude` orchestration system.

At minimum, the implementation must involve the appropriate versions of:

1. Planner
2. Architect
3. Database
4. DB Guardian
5. Backend
6. Frontend
7. Code Impact

8. Testing
9. Security
10. Performance
11. Accessibility
12. Reviewer
13. Documentation
14. Version Control

Use Deployment/DevOps when deployment configuration becomes relevant.

Your framework already defines the full-stack workflow with planning, architecture, database, DB Guardian, backend, frontend, impact analysis, testing, security, performance, review, documentation and version-control stages. Follow that workflow rather than bypassing it.

4. READ REQUIRED CONTEXT BEFORE IMPLEMENTATION

Before coding, read the applicable `.claude` contexts, especially:

- `contexts/project.md`
- `contexts/tech-stack.md`
- `contexts/architecture.md`
- `contexts/database.md`
- `contexts/api-conventions.md`
- `contexts/business-rules.md`
- `contexts/design-system.md`
- `contexts/coding-standards.md`
- `contexts/folder-structure.md`
- `contexts/environment.md`

Also read the applicable:

- agents
- workflows
- skills
- governance/policy documents

Do not invent framework conventions when the repository already defines them.

The existing framework explicitly uses context inheritance such as `tech-stack → design-system/database/api-conventions`, so respect those relationships.

5. BEFORE WRITING CODE

Create an implementation plan internally using the Planner and Architect.

The plan must identify:

- existing code that can be reused
- files to modify
- files to create
- database changes
- API/backend changes
- frontend changes
- security implications
- test requirements
- migration requirements
- dependency impact
- deployment implications

Then begin implementation.

Do not wait for me to approve an ordinary implementation plan.

6. COMPLETE REAK PRODUCT

Build the following three applications as one coherent system.

A. PUBLIC WEBSITE

Routes should include where applicable:

- /
- /about
- /leadership
- /members
- /members/:slug
- /membership
- /membership/apply
- /verify-member
- /news
- /news/:slug
- /notices
- /notices/:slug
- /events

- /events/:slug
 - /resources
 - /contact
 - /properties
 - /properties/:slug
 - /login
 - /forgot-password
 - /reset-password
 - /privacy
 - /terms
-

7. PUBLIC HOMEPAGE

Build a premium, professional REAK association landing page.

Sections:

1. Header
2. Hero
3. REAK introduction
4. Mission / Vision
5. Association benefits
6. Member Property Exchange explanation
7. How the network works
8. Verified members
9. News
10. Notices
11. Events
12. Membership CTA
13. Public property section behind feature flag
14. Contact CTA
15. Footer

Do not invent official REAK content.

Use configurable CMS content.

If real content is unavailable, create elegant empty/configuration states rather than fake content.

8. DESIGN SYSTEM

Create a consistent design system.

Use:

- typography scale
- spacing system
- colors
- buttons
- inputs
- cards
- dialogs
- tables
- badges
- alerts
- navigation
- tabs
- dropdowns
- pagination
- loading states
- error states
- empty states

The design must feel:

- authoritative
- professional
- premium
- modern
- trustworthy
- clean
- Nepal-relevant
- association-focused

Avoid:

- generic AI SaaS aesthetics
- excessive gradients
- excessive rounded cards
- fake luxury imagery
- fake statistics
- fake testimonials
- unnecessary animations

Use existing `ui-ux-pro-max`, `21st.dev` and `framer-motion` capabilities where appropriate. The framework specifically supports these design/component/animation skills.

Do not add animations merely for decoration.

9. MEMBER PORTAL

Create a secure authenticated portal.

Core routes:

- /portal/dashboard
- /portal/properties
- /portal/properties/new
- /portal/properties/:id
- /portal/properties/:id/edit
- /portal/my-properties
- /portal/demands
- /portal/demands/new
- /portal/demands/:id
- /portal/demands/:id/edit
- /portal/matches
- /portal/matches/:id
- /portal/collaborations
- /portal/collaborations/:id
- /portal/members
- /portal/members/:id
- /portal/saved
- /portal/notifications
- /portal/profile
- /portal/organization
- /portal/settings

All portal routes must be:

- authenticated
- permission protected
- RLS protected
- noindex

10. MEMBER DASHBOARD

Database-backed dashboard.

Include:

- Active Properties
- Draft Properties
- Active Requirements

- Potential Matches
- Collaboration Requests
- Saved Properties
- Expiring Items
- Recent Properties
- Quick Actions

Do not display fake numbers.

11. AUTHENTICATION

Implement real authentication.

Support:

- login
- logout
- password reset
- invitation
- account activation
- suspended accounts
- session handling
- protected routes

Never create:

- fake login
 - frontend-only authentication
 - hardcoded credentials
 - insecure bypasses
-

12. MULTI-TENANT MEMBER MODEL

Implement:

- profiles
- member_entities
- entity_users
- invitations
- membership_applications
- roles
- permissions
- role_permissions

- user/profile role assignments

Support:

System-level

- Super Administrator
- Association Administrator

Organization-level

- Member Administrator
- Member Staff

A system-level administrator does not need to belong to a member organization.

Do not use a simple `is_admin` boolean as the primary authorization mechanism.

13. RBAC

Implement permission-based authorization.

Examples:

- members.read
- members.create
- members.update
- members.suspend
- listings.read
- listings.create
- listings.update
- listings.moderate
- demands.read
- demands.create
- matches.read
- match_rules.manage
- collaboration.create
- collaboration.read
- cms.manage
- settings.manage
- audit.read

Use secure server-side permission checks.

Frontend permissions are for UX only.

Database permissions/RLS are authoritative.

14. PROPERTY EXCHANGE

Build the central shared REAK property marketplace/network.

Support:

- search
- filtering
- sorting
- pagination
- saved filters
- saved properties
- property cards
- property detail
- member attribution
- visibility rules

Filters should include where relevant:

- property type
- subtype
- purpose
- province
- district
- municipality
- ward
- locality
- price
- area
- road width
- bedrooms
- bathrooms
- parking
- furnishing
- facing
- amenities
- member
- status
- expiry

Use server-side filtering.

Do not download the entire database to the browser.

15. PROPERTY CREATION

Create a multi-step listing workflow:

1. Type
2. Basic Information
3. Location
4. Specifications
5. Price
6. Amenities
7. Photos
8. Documents
9. Description
10. Contact/Collaboration
11. Visibility
12. Expiry
13. Review
14. Submit

Support:

- Save Draft
- Continue
- Back
- Submit
- validation
- upload progress
- error recovery
- unsaved changes

16. PROPERTY DATABASE

Use structured relational fields.

Include:

- UUID
- reference code
- member entity
- creator
- updater
- title
- property type

- subtype
- purpose
- location
- landmark
- coordinates where appropriate
- currency
- price
- negotiable
- land area
- built-up area
- area unit
- road access
- road width
- road type
- facing
- bedrooms
- bathrooms
- floors
- parking
- furnishing
- description
- internal notes
- visibility
- moderation status
- lifecycle status
- approval
- expiry
- timestamps
- archive/delete state

Use relational tables for:

- listing_media
- listing_documents
- listing_amenities
- listing_contacts
- listing_visibility_members

17. PROPERTY CONTACT PRIVACY

Owner contact must remain separate.

`listing_contacts`

must never be returned by normal property queries.

Protect it using RLS.

Default:

Only the owning organization can access it.

Contact disclosure happens only through explicit collaboration authorization.

18. PROPERTY MEDIA

Separate:

Public/visible listing images

from:

Private documents

Images follow listing visibility.

Documents require signed/private access.

Only authorized users can upload/delete.

Do not assume document types such as citizenship, Lalpurja or PAN unless configured by Admin.

19. DEMANDS / REQUIREMENTS

Build client requirement management.

Support:

- buyer
- tenant
- investor
- other configurable purposes

A member can create requirements for clients while keeping client identity private.

Create:

- demands

- demand_property_types
- demand_locations
- demand_amenities
- demand_contacts
- demand_visibility_members

Do not use arrays as the primary relationship model.

20. DEMAND SEARCH

Allow members to search/filter compatible requirements where visibility permits.

Do not expose:

- client phone
- client email
- client identity
- confidential notes

unless explicitly authorized.

21. NEPAL LOCATION HIERARCHY

Implement:

Province → District → Municipality → Ward → Locality

Use normalized IDs.

Admin can:

- add
- edit
- disable
- reorder

locations.

Do not use uncontrolled duplicated strings as the primary relationship.

22. LAND AREA UNITS

Support:

- Ropani
- Aana
- Paisa
- Dam
- Bigha
- Kattha
- Dhur
- Square Feet
- Square Metres

Automatic conversion must remain disabled unless configured and approved.

Do not invent conversion coefficients.

23. MATCHING ENGINE

Build bidirectional matching:

Demand → Property

and:

Property → Demand

Matching must be server-side.

Create:

- match_rule_sets
- match_rules
- matches
- match_components
- match_actions

Matching must be:

- explainable
- deterministic
- configurable
- auditable

24. NO HARDCODED MATCHING WEIGHTS

Do NOT invent production scoring such as:

- 30% location
- 30% budget
- 20% area

or any other arbitrary weights.

Admin configures rules.

If there is no active published ruleset:

Do not generate score 0 matches.

Show:

Matching is not yet configured by REAK.

Only a valid published ruleset can generate production matches.

25. MATCH EXPLANATION

Show:

- score
- rule set/version
- matching criteria
- partial matches
- failed criteria
- missing data
- price difference
- area difference
- location compatibility
- calculation timestamp

Actions:

- shortlist
- dismiss
- reopen
- request collaboration

- report incorrect data
-

26. COLLABORATION

Create:

collaboration_requests

and:

collaborations

A request can be:

- pending
- accepted
- declined
- cancelled

After acceptance, create the collaboration workspace.

27. COLLABORATION WORKSPACE

Support:

- participants
- messages
- files
- notes
- tasks
- viewings
- activity history

Only authorized participants can access the workspace.

28. CONTACT DISCLOSURE

Never automatically expose private contacts merely because collaboration is accepted.

Create explicit:

collaboration_contact_disclosures

Track:

- collaboration
- data type
- granting organization
- receiving organization
- granting user
- timestamp
- revocation
- policy/version

Enforce with RLS.

29. NOTIFICATIONS

Implement in-app notifications for:

- matches
- collaboration requests
- collaboration acceptance
- collaboration decline
- messages
- listing approval
- listing rejection
- expiry
- invitations
- account events
- association notices

Support:

- unread
 - read
 - mark all read
-

30. ADMIN PORTAL

Build a full administration interface.

Routes should cover:

- dashboard
- members
- users
- invitations
- roles
- permissions
- membership applications
- properties
- demands
- matches
- match rules
- collaborations
- property types
- amenities
- locations
- units
- currencies
- committee
- pages
- news
- notices
- events
- resources
- media
- notifications
- feature flags
- reports
- audit logs
- settings
- security

31. ADMIN CMS

Admin must manage:

- homepage
- about
- mission
- vision
- leadership
- members
- news
- notices

- events
- resources
- membership
- contact
- footer
- navigation
- SEO
- legal pages

Content lifecycle:

- Draft
- Review
- Published
- Archived

32. FEATURE FLAGS

Implement configurable flags:

- public_properties_enabled
- public_member_directory_enabled
- open_registration_enabled
- membership_application_enabled
- property_moderation_required
- matching_enabled
- collaboration_enabled
- deal_tracking_enabled
- auto_unit_conversion
- sms_notifications
- whatsapp_notifications
- email_notifications
- member_export_enabled

Use conservative defaults.

33. PUBLIC PROPERTY SECURITY

Anonymous users must NEVER directly query the private listings table.

Use a sanitized public projection/function/RPC.

Expose only explicitly approved public fields.

Public listing access must require:

- public properties enabled
- active listing
- approved
- public visibility
- active member
- not deleted
- not expired

Same principle for public member profiles.

34. RLS

RLS is mandatory.

Enable RLS on every private table.

Test direct database/API access.

Users must not be able to bypass security by calling APIs directly.

Suspended users must lose access even if they have an existing session.

35. STORAGE SECURITY

Separate storage for:

- CMS public media
- member public assets
- listing media
- private listing documents
- membership documents
- collaboration files

Storage permissions must mirror application authorization.

Do not expose private storage paths publicly.

36. AUDIT LOGGING

Audit:

- login/security changes where possible
- role changes
- member changes
- listing changes
- demand changes
- moderation
- collaboration
- contact disclosure
- feature flag changes
- configuration changes

Do NOT blindly store entire sensitive records in audit logs.

Never duplicate private contact information unnecessarily.

Audit logs are append-only.

Normal users cannot modify/delete them.

37. SECURITY FUNCTIONS

For PostgreSQL `SECURITY DEFINER` functions:

- fixed search_path
- explicit permissions
- strict parameter validation
- no privilege escalation
- no arbitrary SQL
- minimal privileges

Review every security-sensitive function.

38. DATABASE QUALITY

Use:

- UUIDs
- foreign keys

- indexes
- unique constraints
- check constraints
- appropriate nullability
- timestamps
- archive/soft-delete strategy

Reference numbers must be generated server-side/database-side.

39. NO FAKE BUSINESS DATA

Never invent production:

- members
- property listings
- client requirements
- transactions
- executive names
- phone numbers
- emails
- addresses
- statistics
- testimonials
- sponsors
- legal claims

If UI requires content:

Use configuration/empty states.

40. RESPONSIVE DESIGN

The application must work on:

- desktop
- laptop
- tablet
- mobile

Do not simply shrink desktop tables.

Use:

- mobile cards

- filter drawers
- responsive forms
- responsive navigation
- touch-friendly actions

Test actual viewport behavior.

41. ACCESSIBILITY

Target WCAG 2.2 AA.

Verify:

- semantic HTML
- keyboard navigation
- focus states
- labels
- error messages
- contrast
- dialogs
- screen-reader accessibility
- reduced motion

Use the existing Accessibility skill.

The framework already defines Accessibility as a specialized quality function, including contrast validation.

42. PERFORMANCE

Implement:

- lazy loading
- code splitting
- pagination
- indexed queries
- image optimization
- debounced search
- caching where appropriate
- skeleton states
- error boundaries

Do not optimize blindly.

Measure first.

43. SEO

Public pages:

- metadata
- canonical
- Open Graph
- sitemap
- robots
- semantic markup

Private routes:

- noindex
-

44. TESTING

Create a serious test suite.

At minimum:

Unit

- validation
- utilities
- matching rules
- permission logic

Integration

- authentication
- database
- RLS
- API/business logic
- storage

E2E

Test:

1. Admin creates member
2. Admin invites user
3. User activates account
4. User logs in

5. User creates property
 6. Property persists
 7. User edits property
 8. Another member sees permitted property
 9. Private contact remains private
 10. User creates demand
 11. Demand persists
 12. Matching rules are published
 13. Match is generated
 14. Match explanation appears
 15. Collaboration request is sent
 16. Collaboration is accepted
 17. Collaboration workspace opens
 18. Contact remains private
 19. Explicit contact disclosure works
 20. Suspended user loses access
 21. Public feature flag blocks/enables public property visibility
 22. Admin CMS changes appear publicly
-

45. SECURITY TESTING

Explicitly test attempts to:

- access another organization's property
- modify another organization's property
- read private contacts
- read private documents
- access another organization's demands
- modify roles
- grant permissions
- access Admin routes
- access collaboration without participation
- access disabled features
- bypass frontend restrictions using direct API calls
- access data after suspension

Fix all security failures before completion.

46. CODE QUALITY

Follow the repository's existing coding standards.

Use:

- strict TypeScript
- clear naming
- small focused modules
- reusable components
- schema validation
- typed APIs
- error handling
- logging where appropriate
- no dead code
- no unnecessary duplication

Do not leave:

- TODO placeholders for core features
- fake API responses
- simulated database data
- `any` abuse
- hardcoded secrets
- commented-out production code

47. ENVIRONMENT VARIABLES

Never hardcode:

- API keys
- passwords
- Supabase secrets
- OAuth secrets
- tokens

Use environment variables.

Never commit `.env` secrets.

Create/update:

`.env.example`

with variable names only.

48. MIGRATIONS

All database changes must use versioned migrations.

Never manually create undocumented production schema.

Every migration must be:

- deterministic
- reviewable
- reversible where practical
- tested
- protected by DB Guardian review

Do not drop existing data without explicit justification.

49. EXISTING CODE IMPACT

Before major changes, use the Code Impact Agent.

Identify:

- dependencies
- affected routes
- affected components
- affected database objects
- affected APIs
- tests
- documentation

Do not break existing functionality accidentally.

50. GIT / VERSION CONTROL

Use the existing Version Control workflow.

Before meaningful commits:

- inspect git status
- review diff
- ensure no secrets
- ensure unrelated files were not modified

- ensure tests pass

Use meaningful commits.

Do not make giant meaningless commits containing unrelated changes.

Do not push to remote unless repository policy/configuration explicitly allows it or I request it.

51. DOCUMENTATION

Maintain:

- README
- architecture documentation
- database documentation
- environment documentation
- setup instructions
- development instructions
- authentication instructions
- deployment instructions
- security documentation
- API documentation where applicable
- testing instructions

Documentation must reflect the actual implementation.

Do not document features that do not exist.

52. ERROR HANDLING

Every important user-facing operation needs:

- loading state
- success state
- empty state
- validation errors
- server errors
- retry/recovery where appropriate

Do not expose raw database errors to end users.

Log technical details appropriately.

53. OBSERVABILITY

Implement sensible logging around:

- authentication failures
- authorization failures
- important business actions
- matching
- collaboration
- security events
- background failures

Never log:

- passwords
 - access tokens
 - private contact data unnecessarily
 - secrets
-

54. PRODUCTION READINESS

Before declaring completion, verify:

Frontend

- build succeeds
- no TypeScript errors
- no console errors
- no broken routes
- responsive
- accessible

Backend

- validation
- error handling
- authorization
- logging

Database

- migrations
- indexes
- constraints
- RLS

- functions

Security

- authentication
- RBAC
- tenant isolation
- PII isolation
- storage policies
- audit

Testing

- unit
- integration
- E2E
- security

Performance

- critical queries
- property search
- demand search
- matching
- images

55. DO NOT STOP PREMATURELY

Do not stop after:

- creating the homepage
- creating mock dashboards
- creating database tables
- creating routes
- creating forms
- creating UI placeholders

The application is complete only when the feature is actually connected end-to-end.

For example:

A property feature is NOT complete until:

UI → validation → backend → database → RLS → storage → search → detail page → permissions → tests

all work together.

56. USE REAL DATA FLOW

Avoid:

```
button → console.log()
```

Avoid:

```
button → fake success toast
```

Avoid:

```
API → hardcoded JSON
```

Avoid:

```
dashboard → fake numbers
```

Use real application state and database persistence.

57. IMPLEMENTATION ORDER

Use this order unless the existing architecture requires a safer alternative.

Stage 1

Repository audit + architecture confirmation.

Stage 2

Design system + public website foundation.

Stage 3

Database + migrations + RLS + RBAC.

Stage 4

Authentication + member organizations.

Stage 5

Member portal.

Stage 6

Property Exchange.

Stage 7

Demand/Requirement system.

Stage 8

Matching engine.

Stage 9

Collaboration.

Stage 10

Notifications.

Stage 11

Admin + CMS.

Stage 12

Feature flags + reports + audit.

Stage 13

Security hardening.

Stage 14

Performance + accessibility + responsive QA.

Stage 15

Full regression testing.

Stage 16

Documentation + production readiness.

58. CONTINUOUS VALIDATION

After each major stage:

1. run type checks
2. run lint
3. run relevant tests
4. inspect generated code
5. inspect database migrations
6. inspect RLS
7. inspect git diff
8. fix failures
9. continue

Do not carry known failures into later stages unless technically unavoidable and documented.

59. VISUAL QA

For important UI screens, verify actual rendered output.

Check:

- spacing
- typography
- alignment
- hierarchy
- responsive behavior
- loading states
- empty states
- error states
- accessibility
- consistency

Do not judge UI only from source code.

60. FINAL SECURITY AUDIT

Before completion, run the Security Agent.

It must review:

- authentication
- authorization
- RBAC
- RLS
- tenant isolation
- public projections
- contact privacy
- storage
- audit logs
- SECURITY DEFINER functions
- environment variables
- dependency vulnerabilities
- common OWASP risks

Fix discovered vulnerabilities.

61. FINAL DATABASE AUDIT

Run DB Guardian.

Verify:

- migrations
- foreign keys
- indexes
- constraints
- RLS
- policies
- functions
- security
- destructive operations
- data integrity

Fix all critical findings.

62. FINAL CODE REVIEW

Run Reviewer.

Review:

- architecture
- maintainability
- duplication
- correctness
- security
- performance
- testing
- accessibility
- documentation

Fix actionable issues.

63. FINAL REPORT

When the implementation is genuinely complete, provide:

REAK PRODUCTION BUILD REPORT

1. Project Structure

Explain the final architecture.

2. Completed Features

List all implemented features.

3. Routes

List all routes.

4. Database

List:

- tables
- migrations
- functions
- indexes

- RLS policies

5. Authentication

Explain authentication flow.

6. RBAC

Explain system-level and organization-level roles.

7. Security

Explain:

- RLS
- tenant isolation
- PII protection
- public projections
- storage
- audit

8. Matching

Explain matching implementation and rule configuration.

9. Collaboration

Explain collaboration and contact disclosure.

10. Testing

Report:

- unit tests
- integration tests
- E2E tests
- security tests
- accessibility tests

Include passed/failed counts.

11. Performance

Report important performance findings.

12. Accessibility

Report WCAG-related validation.

13. CMS

Report CMS functionality.

14. Environment

List required environment variable NAMES only.

Never reveal secret values.

15. Deployment

Explain production deployment steps.

16. Remaining Configuration

List only information that REAK itself must provide, such as:

- official logo
- official legal name
- official colors
- actual committee information
- actual contact information
- actual membership policies
- approved matching rules
- approved area conversion rules

17. Known Limitations

Be honest.

18. Final Status

Use exactly one of:

READY FOR PRODUCTION

or

NOT READY FOR PRODUCTION

If NOT READY, list the exact blockers.

64. IMPORTANT

You are not being asked to merely produce a prototype.

Build a serious production-quality application.

Prioritize:

1. Security
2. Correctness
3. Data integrity
4. Real functionality
5. Maintainability
6. Accessibility
7. Performance
8. Visual quality

Do not sacrifice security for convenience.

Do not sacrifice functionality for visual polish.

Do not sacrifice maintainability for speed.

65. START NOW

Begin by inspecting the repository.

Read the `.claude` framework and relevant contexts.

Run the Planner/Architect workflow.

Then implement REAK end-to-end.

Do not ask me to repeat the specification.

Do not ask me what technology the project uses until you have inspected the repository.

Do not generate fake production data.

Do not stop at a mockup.

Do not stop at Phase 1.

Do not stop after creating the database.

Do not stop after creating the frontend.

Continue until the complete integrated REAK platform is implemented, tested, secured, reviewed and documented.

BEGIN DEVELOPMENT.